

Escalabilidade do código de Navier-Stokes no NPAD

Matheus Marinho

20 de setembro de 2026

O que foi medido

O ponto de partida é o código da Tarefa 11: difusão viscosa em malha $n \times n$, diferenças centrais, Euler explícito, bordas periódicas. Lá o objetivo era comparar cláusulas e nada foi otimizado. Aqui o objetivo é o oposto — encontrar os gargalos e removê-los em versões sucessivas.

As três definições usadas:

termo	definição
escalável	dá para manter a eficiência se o problema crescer junto com o número de threads
fortemente escalável	mantém a eficiência aumentando as threads sem aumentar o problema
fracamente escalável	mantém a eficiência se o problema crescer na mesma taxa das threads

Fortemente escalável implica escalável, e fracamente escalável também. Nenhuma das duas implica a outra.

Na escalabilidade forte, $n = 2048$ fixo e as threads vão de 1 a 128; eficiência = speedup / p. Na fraca, o trabalho por thread fica constante em 1024×1024 células: $p = 1, 4, 16, 64$ com $n = 1024, 2048, 4096, 8192$; eficiência = $T(1) / T(p)$.

Máquina. NPAD, partição amd-512, nó r2n19 com `--exclusive`: dois AMD EPYC 7713, 64 núcleos cada, **8 domínios NUMA** de 16 núcleos e **32 MB de L3 por CCX de 8 núcleos** — 512 MB de L3 somando os 16 CCX. gcc 8.5.0 com `-O2 -Wall -Wextra -std=c11 -fopenmp, OMP_PLACES=cores, OMP_PROC_BIND=close, clock_gettime(CLOCK_MONOTONIC)`, mediana de 5 execuções, 200 passos.

Essa topologia é o que explica quase tudo o que vem a seguir, então vale fixar um número: com $n = 2048$, as quatro matrizes ocupam **128 MB**.

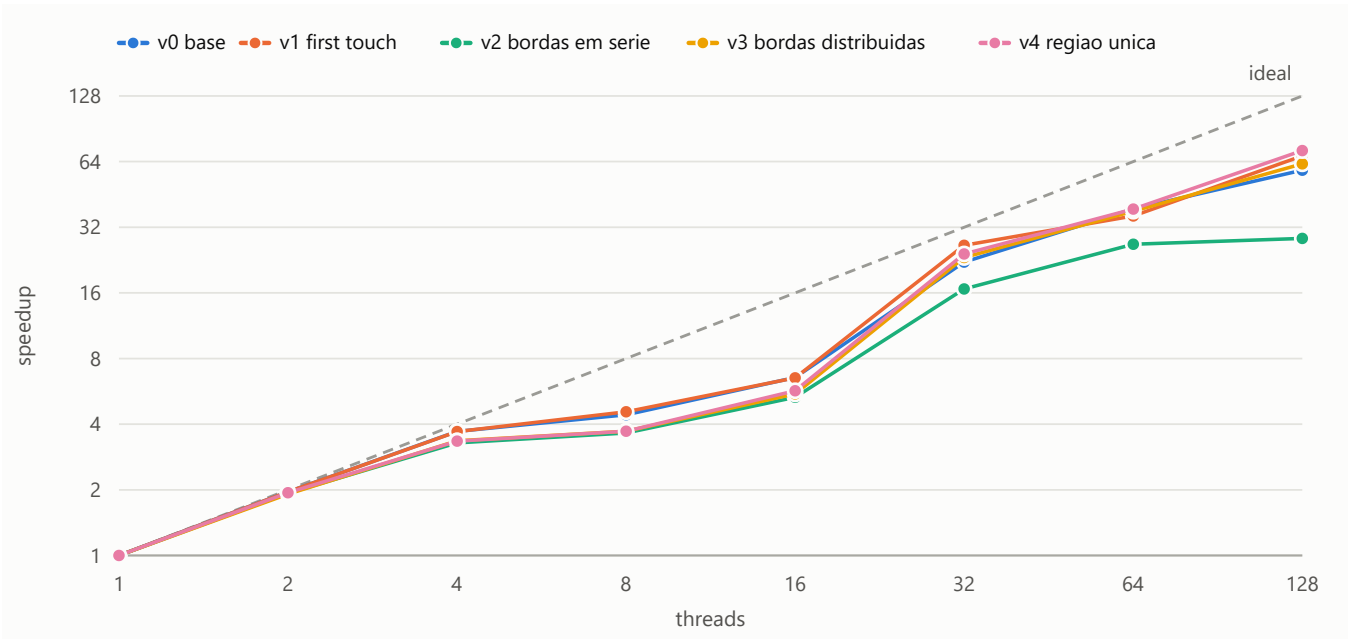
As cinco versões

mudança	gargalo atacado
v0 código da Tarefa 11	— (referência)
v1 inicialização paralela, com o mesmo <code>schedule(static)</code> do cálculo	<i>first touch</i> : as páginas passam a ficar no nó NUMA da thread que as usa
v2 interior sem %, bordas tratadas à parte, em série	4 divisões inteiras por célula
v3 cada thread fecha as bordas laterais das próprias linhas	trecho serial criado pela v2
v4 uma única região paralela em volta do laço de tempo	200 fork-joins

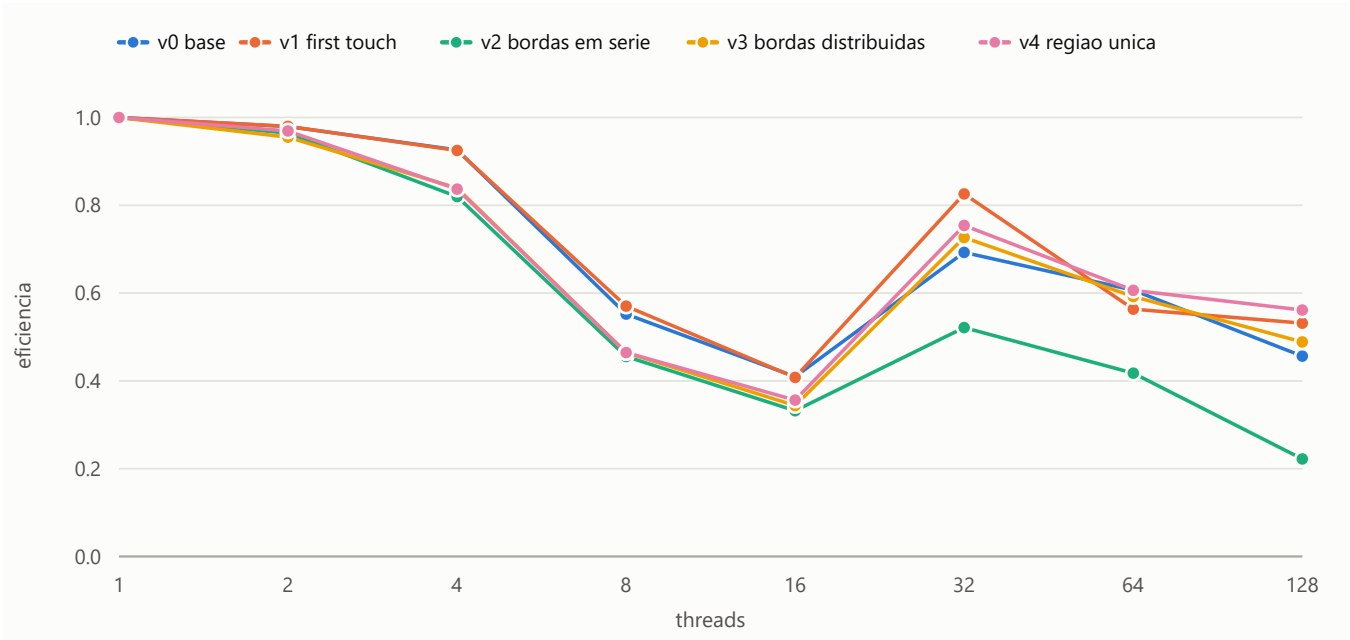
Na v4 os quatro ponteiros entram como `firstprivate`, então cada thread troca a sua própria cópia ao fim do passo. A barreira implícita do `omp for` já garante que ninguém comece o passo seguinte antes de todos terminarem, e por isso não é preciso `single` nem barreira explícita para a troca.

Todas as versões foram comparadas byte a byte com a v0 sequencial, em 1, 8 e 64 threads. As quinze combinações deram campo final idêntico.

Escalabilidade forte



Speedup com n = 2048 fixo. Escala log-log: a reta tracejada é o speedup ideal.



Eficiência = speedup / p, mesmo experimento. A queda até 16 threads e a recuperação em 32 aparecem nas cinco versões.

Tempos em segundos:

threads	v0	v1	v2	v3	v4
1	4,4549	4,4659	3,5188	3,5310	3,5507
2	2,2740	2,2791	1,8241	1,8487	1,8316
4	1,2025	1,2072	1,0727	1,0519	1,0610
8	1,0085	0,9789	0,9655	0,9502	0,9555
16	0,6798	0,6841	0,6628	0,6420	0,6233
32	0,2010	0,1690	0,2110	0,1519	0,1472
64	0,1147	0,1239	0,1317	0,0932	0,0915
128	0,0763	0,0656	0,1236	0,0564	0,0494

Eficiência:

threads	v0	v1	v2	v3	v4
2	0,98	0,98	0,96	0,96	0,97
8	0,55	0,57	0,46	0,46	0,46
16	0,41	0,41	0,33	0,34	0,36
32	0,69	0,83	0,52	0,73	0,75
128	0,46	0,53	0,22	0,49	0,56

A curva não é monótona, e isso não é ruído. A eficiência cai até 16 threads, sobe em 32 e volta a cair. As cinco versões fazem o mesmo desenho, o que descarta acaso. A explicação está na topologia: com ligação `close`, as threads ocupam CCX consecutivos, e o L3 agregado cresce com elas.

threads	CCX ocupados	L3 agregado	conjunto de trabalho
8	1	32 MB	128 MB
16	2	64 MB	128 MB
32	4	128 MB	128 MB
128	16	512 MB	128 MB

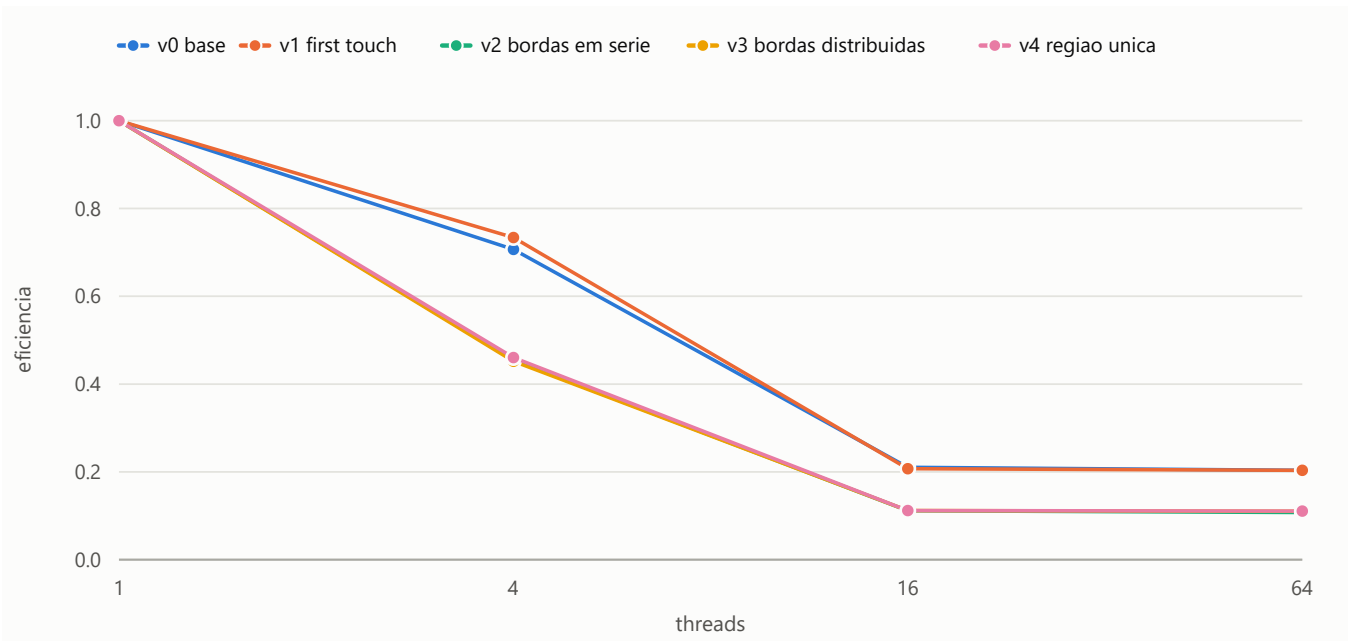
Em 32 threads o L3 agregado alcança o tamanho do problema, os dados passam a sobreviver de um passo para o outro e o tráfego para a memória despenca. É um ganho superlinear: dobrando as threads de 16 para 32, o tempo cai 3,4× na `v0` e 4,2× na `v4`. Conferindo pela banda: em 8 threads são 25,6 GB em 1,01 s, ou ~25 GB/s, valor de DRAM de um nó NUMA só; em 128 threads, os mesmos 25,6 GB em 0,0494 s dão ~518 GB/s, acima de qualquer DRAM e compatível com dados servidos pelo L3.

A `v2` piorou o programa paralelo, e esse é o resultado mais útil. Tirar o % deixou a versão sequencial 1,27× mais rápida (4,455 s → 3,519 s), o que era o esperado. Mas em 128 threads a `v2` ficou 1,6× **mais lenta** que a `v0`. O motivo é que eu deixei `bordas()` em série: ela percorre as colunas 0 e n-1 de 2046 linhas, ou seja, 4092 acessos dispersos, um por linha de cache, executados por uma thread só. Pela lei de Amdahl, um speedup preso em 28,5× com 128 threads corresponde a uma fração serial de cerca de 2,8%. Distribuir essas bordas (`v3`) devolveu 2,19× em 128 threads.

Ganhos em 128 threads: a `v1` é 14% mais rápida que a `v0` — o *first touch* só rende quando há mais de um nó NUMA envolvido, o que explica por que ele não aparece abaixo de 32 threads; a `v3` é 2,19× mais rápida que a `v2`; e a `v4` ganha mais 1,14× sobre a `v3` ao eliminar os 200 fork-joins. Do sequencial da `v0` até a `v4` em 128 threads, 4,455 s → 0,0494 s: **90×**.

Veredito: o código não é fortemente escalável. A eficiência sai de 1,00 e termina em 0,56 com 128 threads na melhor versão. Mantê-la constante com o problema fixo não acontece em nenhuma delas.

Escalabilidade fraca



Trabalho por thread constante em 1024 × 1024 células. Eficiência = T(1)/T(p).

p	n	v0	v1	v2	v3	v4
1	1024	1,00	1,00	1,00	1,00	1,00
4	2048	0,71	0,73	0,45	0,45	0,46
16	4096	0,21	0,21	0,11	0,11	0,11
64	8192	0,20	0,20	0,11	0,11	0,11

Tempos no maior ponto (p = 64, n = 8192): 4,109 s na **v0**, 4,119 s na **v1**, 4,245 s na **v2**, 4,108 s na **v3** e 4,097 s na **v4**. **As versões convergem.** Com 2 GB de matrizes contra 512 MB de L3, nada mais cabe em cache, o programa passa a depender só da banda de memória, e nenhuma das otimizações toca nesse limite: 410 GB de tráfego em 4,10 s dão ~100 GB/s.

As versões otimizadas exibem eficiência *pior* porque o denominador mudou: o T(1) delas é 1,85× mais rápido (0,453 s contra 0,837 s), então a mesma perda absoluta vira uma queda relativa maior. Comparar eficiência entre versões com bases diferentes engana; os tempos absolutos são a comparação honesta, e por eles a **v4** é a melhor no maior ponto, com a **v3** praticamente empatada.

Veredito: o código não é fracamente escalável. A eficiência cai para 0,11.

Uma ressalva de rigor: falhar nesse teste não prova, sozinho, que o programa não é escalável em nenhuma taxa de crescimento — só na taxa que define a escalabilidade fraca. Mas como o limite é banda de memória, que não cresce com **n**, crescer o problema mais rápido só pioraria.

Gargalos identificados

1. **Capacidade de L3 agregado** — o de maior efeito, e o que eu não tinha visto na Tarefa 11. Enquanto o conjunto de trabalho não cabe no L3 somado das threads, o programa fica preso na DRAM. É o que faz a curva de eficiência mergulhar até 16 threads e se recuperar em 32.
2. **Trecho serial nas bordas** — criado por mim na **v2**. 2,8% de fração serial bastaram para limitar o speedup a 28,5× em 128 threads.
3. **NUMA / first touch** — vale ~14% em 128 threads, e nada abaixo de 32, onde todas as threads estão no mesmo nó NUMA de qualquer forma.
4. **Fork-join por passo** — 200 criações de equipe, ~1,14× em 128 threads.
5. **Banda de memória** — o limite de fundo. Quando o problema não cabe no cache, as cinco versões dão o mesmo tempo.

Conclusão

O programa **não é fortemente nem fracamente escalável**. Ele é, em termos absolutos, muito mais rápido: 90× do sequencial original até a **v4** em 128 threads, sendo 1,27× de código sequencial melhor e o resto de paralelismo.

O achado que eu não esperava é que, para este código, a eficiência **melhora** quando se adicionam threads em uma faixa (16 → 32), porque o cache agregado cresce junto. Isso inverte a intuição de que mais threads sempre diluem a eficiência, e sugere que a forma certa de dimensionar este programa não é crescer o problema na mesma taxa das threads, e sim mantê-lo dentro do L3 agregado disponível — ou seja, crescer mais devagar do que a escalabilidade fraca supõe.

Código

escalabilidade.c · 392 linhas

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <malloc.h>
4 #include <math.h>
5 #include <omp.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9 #include <time.h>
10
11 typedef struct {
12     int n;
13     double h;
14     double viscosidade;
15     double dt;
16 } Malha;
17
18 typedef void (*Inicializador)(const Malha *, double *, double *, double *, double *);
19 typedef void (*Simulador)(const Malha *, double **, double **, double **, double **, int);
```

```

20
21 typedef struct {
22     const char *nome;
23     const char *descricao;
24     Inicializador inicializa;
25     Simulador simula;
26 } Versao;
27
28 static double agora(void)
29 {
30     struct timespec t;
31     clock_gettime(CLOCK_MONOTONIC, &t);
32     return (double)t.tv_sec + (double)t.tv_nsec * 1e-9;
33 }
34
35 static double *aloca(int n)
36 {
37     double *p = malloc((size_t)n * (size_t)n * sizeof(double));
38     if (p == NULL) {
39         fprintf(stderr, "memoria insuficiente para malha %d\n", n);
40         exit(EXIT_FAILURE);
41     }
42     return p;
43 }
44
45 static inline double fator_de(const Malha *m)
46 {
47     return m->viscosidade * m->dt / (m->h * m->h);
48 }
49
50 static inline void celula_periodica(const Malha *m,
51                                   const double *u, const double *v,
52                                   double *un, double *vn, int i, int j)
53 {
54     const int n = m->n;
55     const int i_ant = (i + n - 1) % n;
56     const int i_prox = (i + 1) % n;
57     const int j_ant = (j + n - 1) % n;
58     const int j_prox = (j + 1) % n;
59     const int centro = i * n + j;
60     const double fator = fator_de(m);
61
62     const double laplaciano_u = u[i_prox * n + j] + u[i_ant * n + j]
63                               + u[i * n + j_prox] + u[i * n + j_ant]
64                               - 4.0 * u[centro];
65     const double laplaciano_v = v[i_prox * n + j] + v[i_ant * n + j]
66                               + v[i * n + j_prox] + v[i * n + j_ant]
67                               - 4.0 * v[centro];
68
69     un[centro] = u[centro] + fator * laplaciano_u;
70     vn[centro] = v[centro] + fator * laplaciano_v;
71 }
72
73 static void inicializa_sequencial(const Malha *m, double *u, double *v,
74                                   double *un, double *vn)
75 {
76     const int total = m->n * m->n;
77     for (int k = 0; k < total; k++) {
78         u[k] = 1.0;
79         v[k] = 0.0;
80         un[k] = 0.0;
81         vn[k] = 0.0;
82     }
83     u[(m->n / 2) * m->n + m->n / 2] += 1.0;
84 }
85
86 static void inicializa_first_touch(const Malha *m, double *u, double *v,
87                                   double *un, double *vn)
88 {
89     const int n = m->n;
90     #pragma omp parallel for schedule(static)
91     for (int i = 0; i < n; i++)
92         for (int j = 0; j < n; j++) {
93             const int k = i * n + j;
94             u[k] = 1.0;
95             v[k] = 0.0;
96             un[k] = 0.0;
97             vn[k] = 0.0;
98         }
99     u[(n / 2) * n + n / 2] += 1.0;
100 }
101
102 static void troca(double **a, double **b)
103 {
104     double *t = *a;
105     *a = *b;
106     *b = t;
107 }
108

```

```

109 static void passo_modulo(const Malha *m, const double *u, const double *v,
110                          double *un, double *vn)
111 {
112     #pragma omp parallel for schedule(static)
113     for (int i = 0; i < m->n; i++)
114         for (int j = 0; j < m->n; j++)
115             celula_periodica(m, u, v, un, vn, i, j);
116 }
117
118 static void bordas(const Malha *m, const double *u, const double *v,
119                  double *un, double *vn)
120 {
121     const int n = m->n;
122     for (int j = 0; j < n; j++) {
123         celula_periodica(m, u, v, un, vn, 0, j);
124         celula_periodica(m, u, v, un, vn, n - 1, j);
125     }
126     for (int i = 1; i < n - 1; i++) {
127         celula_periodica(m, u, v, un, vn, i, 0);
128         celula_periodica(m, u, v, un, vn, i, n - 1);
129     }
130 }
131
132 static inline void linha_interior(const Malha *m, const double *u, const double *v,
133                                  double *un, double *vn, int i)
134 {
135     const int n = m->n;
136     const double fator = fator_de(m);
137
138     const double *u_ant = u + (i - 1) * n;
139     const double *u_cen = u + i * n;
140     const double *u_prox = u + (i + 1) * n;
141     const double *v_ant = v + (i - 1) * n;
142     const double *v_cen = v + i * n;
143     const double *v_prox = v + (i + 1) * n;
144     double *un_cen = un + i * n;
145     double *vn_cen = vn + i * n;
146
147     for (int j = 1; j < n - 1; j++) {
148         const double laplaciano_u = u_prox[j] + u_ant[j]
149                                     + u_cen[j + 1] + u_cen[j - 1]
150                                     - 4.0 * u_cen[j];
151         const double laplaciano_v = v_prox[j] + v_ant[j]
152                                     + v_cen[j + 1] + v_cen[j - 1]
153                                     - 4.0 * v_cen[j];
154         un_cen[j] = u_cen[j] + fator * laplaciano_u;
155         vn_cen[j] = v_cen[j] + fator * laplaciano_v;
156     }
157 }
158
159 static void passo_bordas_seriais(const Malha *m, const double *u, const double *v,
160                                  double *un, double *vn)
161 {
162     #pragma omp parallel for schedule(static)
163     for (int i = 1; i < m->n - 1; i++)
164         linha_interior(m, u, v, un, vn, i);
165
166     bordas(m, u, v, un, vn);
167 }
168
169 static inline void linha_com_bordas(const Malha *m, const double *u, const double *v,
170                                    double *un, double *vn, int i)
171 {
172     const int n = m->n;
173
174     celula_periodica(m, u, v, un, vn, i, 0);
175     celula_periodica(m, u, v, un, vn, i, n - 1);
176
177     if (i > 0 && i < n - 1)
178         linha_interior(m, u, v, un, vn, i);
179     else
180         for (int j = 1; j < n - 1; j++)
181             celula_periodica(m, u, v, un, vn, i, j);
182 }
183
184 static void passo_bordas_distribuidas(const Malha *m, const double *u, const double *v,
185                                       double *un, double *vn)
186 {
187     #pragma omp parallel for schedule(static)
188     for (int i = 0; i < m->n; i++)
189         linha_com_bordas(m, u, v, un, vn, i);
190 }
191
192 static void simula_fork_join(const Malha *m, double **u, double **v,
193                             double **un, double **vn, int passos,
194                             void (*passo)(const Malha *, const double *, const double *,
195                                             const double *, const double *))
196 {
197     for (int t = 0; t < passos; t++) {

```

```

198     passo(m, *u, *v, *un, *vn);
199     troca(u, un);
200     troca(v, vn);
201 }
202 }
203
204 static void simula_v0(const Malha *m, double **u, double **v,
205                     double **un, double **vn, int passos)
206 {
207     simula_fork_join(m, u, v, un, vn, passos, passo_modulo);
208 }
209
210 static void simula_v2(const Malha *m, double **u, double **v,
211                     double **un, double **vn, int passos)
212 {
213     simula_fork_join(m, u, v, un, vn, passos, passo_bordas_seriais);
214 }
215
216 static void simula_v3(const Malha *m, double **u, double **v,
217                     double **un, double **vn, int passos)
218 {
219     simula_fork_join(m, u, v, un, vn, passos, passo_bordas_distribuidas);
220 }
221
222 static void simula_v4(const Malha *m, double **pu, double **pv,
223                     double **pun, double **pvn, int passos)
224 {
225     double *u = *pu, *v = *pv, *un = *pun, *vn = *pvn;
226
227     #pragma omp parallel firstprivate(u, v, un, vn)
228     {
229         for (int t = 0; t < passos; t++) {
230             #pragma omp for schedule(static)
231             for (int i = 0; i < m->n; i++)
232                 linha_com_bordas(m, u, v, un, vn, i);
233
234             troca(&u, &un);
235             troca(&v, &vn);
236         }
237     }
238
239     if (passos % 2 == 1) {
240         troca(pu, pun);
241         troca(pv, pvn);
242     }
243 }
244
245 static const Versao VERSOES[] = {
246     { "v0", "base da tarefa 11", inicializa_sequencial, simula_v0 },
247     { "v1", "first touch paralelo", inicializa_first_touch, simula_v0 },
248     { "v2", "interior sem modulo, bordas em serie", inicializa_first_touch, simula_v2 },
249     { "v3", "bordas distribuidas nas threads", inicializa_first_touch, simula_v3 },
250     { "v4", "uma unica regioao paralela", inicializa_first_touch, simula_v4 },
251 };
252 static const int N_VERSOES = (int)(sizeof(VERSOES) / sizeof(VERSOES[0]));
253
254 static int comparar_double(const void *a, const void *b)
255 {
256     double x = *(const double *)a, y = *(const double *)b;
257     return (x > y) - (x < y);
258 }
259
260 static double mediana(double *v, int n)
261 {
262     qsort(v, (size_t)n, sizeof(double), comparar_double);
263     return v[n / 2];
264 }
265
266 static Malha malha_de(int n)
267 {
268     const double r = 0.20;
269     Malha m;
270     m.n = n;
271     m.h = 1.0 / (double)n;
272     m.viscosidade = 0.01;
273     m.dt = r * m.h * m.h / m.viscosidade;
274     return m;
275 }
276
277 static double executar(const Versao *versao, const Malha *m, int passos,
278                     int execucoes, double *saida)
279 {
280     double *tempos = malloc((size_t)execucoes * sizeof(double));
281     double *u = aloca(m->n), *v = aloca(m->n);
282     double *un = aloca(m->n), *vn = aloca(m->n);
283
284     for (int e = 0; e < execucoes; e++) {
285         double *a = u, *b = v, *c = un, *d = vn;
286         versao->inicializa(m, a, b, c, d);

```

```

287
288     double t0 = agora();
289     versao->simula(m, &a, &b, &c, &d, passos);
290     tempos[e] = agora() - t0;
291
292     if (e == execucoes - 1 && saida != NULL)
293         memcpy(saida, a, (size_t)m->n * (size_t)m->n * sizeof(double));
294 }
295
296 double t = mediana(tempos, execucoes);
297 free(tempos); free(u); free(v); free(un); free(vn);
298 return t;
299 }
300
301 static void verificar(int n, int passos)
302 {
303     Malha m = malha_de(n);
304     const size_t bytes = (size_t)n * (size_t)n * sizeof(double);
305     double *referencia = aloca(n);
306     double *obtido = aloca(n);
307
308     omp_set_num_threads(1);
309     executar(&VERSOES[0], &m, passos, 1, referencia);
310
311     printf("# verificacao: malha %d, %d passos, campo final comparado com v0 sequencial\n", n, passos);
312     const int contagens[] = { 1, 8, 64 };
313     for (int k = 0; k < N_VERSOES; k++)
314         for (int c = 0; c < 3; c++) {
315             omp_set_num_threads(contagens[c]);
316             executar(&VERSOES[k], &m, passos, 1, obtido);
317             printf("# %s com %3d thread(s): %s\n", VERSOES[k].nome, contagens[c],
318                 memcmp(obtido, referencia, bytes) == 0 ? "identico" : "DIFERENTE");
319         }
320
321     free(referencia);
322     free(obtido);
323 }
324
325 static void escalabilidade_forte(int n, int passos, int execucoes, int max_threads)
326 {
327     Malha m = malha_de(n);
328
329     for (int k = 0; k < N_VERSOES; k++) {
330         double t1 = 0.0;
331         for (int p = 1; p <= max_threads; p *= 2) {
332             omp_set_num_threads(p);
333             double t = executar(&VERSOES[k], &m, passos, execucoes, NULL);
334             if (p == 1)
335                 t1 = t;
336             printf("forte,%s,%d,%d,%.6f,%.4f,%.4f\n",
337                 VERSOES[k].nome, p, n, t, t1 / t, (t1 / t) / (double)p);
338             fflush(stdout);
339         }
340     }
341 }
342
343 static void escalabilidade_fraca(int n_base, int passos, int execucoes, int max_threads)
344 {
345     for (int k = 0; k < N_VERSOES; k++) {
346         double t1 = 0.0;
347         for (int p = 1; p <= max_threads; p *= 4) {
348             int fator = 1;
349             for (int q = p; q > 1; q /= 4)
350                 fator *= 2;
351             int n = n_base * fator;
352
353             Malha m = malha_de(n);
354             omp_set_num_threads(p);
355             double t = executar(&VERSOES[k], &m, passos, execucoes, NULL);
356             if (p == 1)
357                 t1 = t;
358             printf("fraca,%s,%d,%d,%.6f,%.4f,%.4f\n",
359                 VERSOES[k].nome, p, n, t, (double)p * t1 / t, t1 / t);
360             fflush(stdout);
361         }
362     }
363 }
364
365 int main(int argc, char *argv[])
366 {
367     const char *modo = (argc > 1) ? argv[1] : "forte";
368     int n = (argc > 2) ? atoi(argv[2]) : 2048;
369     int passos = (argc > 3) ? atoi(argv[3]) : 100;
370     int execucoes = (argc > 4) ? atoi(argv[4]) : 3;
371     int max_threads = (argc > 5) ? atoi(argv[5]) : 128;
372
373     mallopt(M_MMAP_THRESHOLD, 128 * 1024);
374
375     if (strcmp(modo, "verificar") == 0) {

```

```
376     verificar(n, passos);
377     return 0;
378 }
379
380 printf("# versoes:\n");
381 for (int k = 0; k < N_VERSOES; k++)
382     printf("# %s = %s\n", VERSOES[k].nome, VERSOES[k].descricao);
383 printf("# passos=%d execucoes=%d (mediana)\n", passos, execucoes);
384 printf("modo,versao,threads,n,tempo,speedup,eficiencia\n");
385
386 if (strcmp(modo, "fraca") == 0)
387     escalabilidade_fraca(n, passos, execucoes, max_threads);
388 else
389     escalabilidade_forte(n, passos, execucoes, max_threads);
390
391 return 0;
392 }
```